

Healthcare Data Warehouse & Analytics Platform

End-to-End Data Engineering Solution for Hospital Network Analytics

Batch ETL + Real-Time Patient Vitals Monitoring + Clinical BI Dashboards

Key Highlights:

- Synthetic clinical data generated with **Synthea** (realistic patient records)
- **Medallion Architecture** (Bronze/Silver/Gold) on Microsoft Azure
- Real-time **IoT vital signs streaming** with anomaly detection alerts
- **Star Schema** with SCD Type 2 for patient and provider dimensions
- **Power BI dashboards** for hospital utilization, readmission rates, cost analysis
- Data governance awareness and role-based access patterns

Team Size	6 Members
Duration	10 Weeks
Cloud Platform	Microsoft Azure
Data Source	Synthea + Custom Generators + IoT Simulator
Complexity	High — Modeling-heavy + Enterprise DE showcase

Prepared by

Mahmoud Salem

[linkedin.com/in/ma7moudalysalem](https://www.linkedin.com/in/ma7moudalysalem)

Table of Contents

#	Section	Description
1	Business Problem & Objectives	Why this project matters
2	Solution Architecture	Medallion Architecture on Azure
3	Data Sources (6 Sources)	Synthea, IoT, custom generators
4	Milestone 1: Data Collection & EDA	Python + SQL
5	Milestone 2: Data Modeling & Warehouse	Star Schema + SCD + Synapse
6	Milestone 3: Pipelines (Batch + Stream)	ADF + Databricks + Event Hubs
7	Milestone 4: BI Dashboards	Power BI + DAX + Streamlit
8	Milestone 5: DevOps & Presentation	CI/CD + Monitoring + Report
9	Team Role Distribution (6 Members)	Detailed responsibilities
10	Timeline (10 Weeks)	Week-by-week schedule
11	Technology Stack	Complete tools list
12	Deliverables Checklist	Final submission items

1. Business Problem & Objectives

A hospital network operating **5 hospitals** and **20+ clinics** across Egypt faces critical challenges in managing and analyzing their healthcare data. Patient records are scattered across multiple Electronic Health Record (EHR) systems, insurance claims are processed through separate billing platforms, pharmacy data lives in isolated systems, and real-time patient monitoring devices generate continuous vital signs data that is never analyzed historically.

The hospital leadership needs a **unified data analytics platform** to answer critical questions:

- **Patient Outcomes:** What are the readmission rates by condition? Which treatment protocols yield the best outcomes?
- **Operational Efficiency:** What is the average bed occupancy rate? Where are the bottlenecks in patient flow?
- **Financial Analysis:** Which departments generate the most revenue? What is the claim denial rate by payer?
- **Real-Time Monitoring:** Can we detect deteriorating patient conditions early through vital signs anomaly detection?
- **Population Health:** What are the disease prevalence trends across regions? How do demographics correlate with conditions?

Data Privacy Note: This project uses **Synthea** — an open-source synthetic patient generator — to create realistic but entirely fictional clinical data. No real patient data is used. However, the architecture demonstrates data governance best practices (role-based access, data masking patterns, audit logging) that would be required in a production healthcare environment.

2. Solution Architecture

The platform follows the **Medallion Architecture** (Bronze - Silver - Gold) deployed on Microsoft Azure, with a dedicated real-time streaming lane for IoT patient vital signs monitoring.

Layer	Purpose	Azure Services	Data Format
Bronze (Raw)	Ingest raw data as-is, no transformations	Azure Data Lake Gen2	CSV, JSON, Parquet (raw)
Silver (Validated)	Cleaned, deduplicated, standardized, schema-enforced	Databricks (PySpark)	Delta Lake (partitioned)
Gold (Business)	Star schema, aggregated KPIs, BI-ready tables	Azure Synapse Analytics	Synapse dedicated SQL pool
Speed Layer	Real-time vital signs processing with alerts	Event Hubs + Spark Streaming	Delta Lake (append mode)

2.1 Data Flow Overview

Batch Path: Synthea CSV/JSON files → Azure Data Lake (Bronze) → ADF Copy Activities → Databricks PySpark (Silver) → Databricks transforms (Gold) → Synapse Analytics → Power BI

Stream Path: IoT Vital Signs Simulator → Azure Event Hubs → Spark Structured Streaming → Anomaly Detection → Delta Lake + Alerts → Power BI (DirectQuery) / Streamlit Live Dashboard

3. Data Sources

The project uses **6 data sources** — 5 batch sources generated by Synthea and custom Python scripts, plus 1 real-time streaming source simulating IoT patient monitors.

#	Source	Format	Key Fields	Volume
1	Patients	CSV (Synthea)	patient_id, name, birth_date, gender, race, city, county, state, insurance_type	100K patients
2	Encounters	CSV (Synthea)	encounter_id, patient_id, provider_id, encounter_type, start_date, end_date, total_cost, payer_id	500K encounters
3	Conditions	CSV (Synthea)	condition_id, patient_id, encounter_id, ICD10_code, description, onset_date, resolved_date	300K conditions
4	Medications	CSV (Synthea)	medication_id, patient_id, encounter_id, drug_code, description, start_date, stop_date, cost	400K prescriptions
5	Claims & Billing	JSON (Custom)	claim_id, patient_id, encounter_id, payer_name, amount_billed, amount_paid, status, denial_reason	200K claims
6	Vital Signs (IoT)	JSON (Stream)	device_id, patient_id, heart_rate, systolic_bp, diastolic_bp, temperature, spo2, timestamp	Continuous

3.1 About Synthea

Synthea is an open-source patient data simulator maintained by MITRE Corporation. It generates realistic (but entirely fictional) patient histories including demographics, encounters, conditions, medications, procedures, and observations. The data follows real clinical pathways and uses standard medical coding systems (ICD-10, SNOMED, RxNorm). This makes it ideal for building healthcare data engineering projects without any privacy concerns.

Installation: `java -jar synthea-with-dependencies.jar -p 100000 --exporter.csv.export true`

4. Milestone 1: Data Collection, Exploration & Preprocessing

Modules: Python Programming + SQL & Database Management

Duration: Weeks 1-2

4.1 Data Generation & Collection

- **Synthea Generation:** Run Synthea to generate 100K synthetic patients with full medical histories
 - Configure disease modules: diabetes, hypertension, heart disease, respiratory conditions
 - Export as CSV files: patients.csv, encounters.csv, conditions.csv, medications.csv, procedures.csv, observations.csv
- **Custom Claims Generator (Python):** Write `generate_claims.py` using Faker library
 - Generate insurance claims with realistic denial rates (15-20%), multiple payer types
 - Include claim statuses: submitted, approved, denied, appealed, paid
- **IoT Vital Signs Simulator:** Write `simulate_vitals.py`
 - Generate realistic vital signs: heart rate (60-100 normal), BP (120/80 normal), temperature (36.5-37.5C), SpO2 (95-100%)
 - Inject anomalies: tachycardia spikes, hypotension episodes, fever patterns
- **Flask REST API:** Write `provider_api.py` serving provider/doctor data
 - Endpoints: GET /providers, GET /providers/{id}, GET /providers?specialty=cardiology

4.2 Exploratory Data Analysis (EDA)

Create a comprehensive Jupyter Notebook with **15+ visualizations**:

- **Demographics:** Age distribution histogram, gender ratio, geographic distribution (city/state heatmap)
- **Clinical Patterns:** Top 20 conditions by prevalence, condition co-occurrence matrix (heatmap)
- **Encounter Analysis:** Encounter types distribution (pie chart), average encounter duration, encounters per patient
- **Financial:** Cost distribution by encounter type (box plot), payer breakdown, claim denial rate by payer
- **Medications:** Top prescribed drugs (bar chart), medication duration analysis, polypharmacy rates
- **Temporal:** Encounter volume over time (time-series), seasonal patterns, day-of-week patterns
- **Automated Report:** Generate ydata-profiling report for each dataset

4.3 Database Schema Design (PostgreSQL)

Design a **3NF normalized relational schema** in PostgreSQL:

Table	Primary Key	Foreign Keys	Key Columns
patients	patient_id (UUID)	—	name, birth_date, gender, race, city, state, insurance_type
providers	provider_id (UUID)	—	name, specialty, organization, city
encounters	encounter_id (UUID)	patient_id, provider_id	encounter_type, start_date, end_date, total_cost
conditions	condition_id (UUID)	patient_id, encounter_id	icd10_code, description, onset_date, resolved_date
medications	medication_id (UUID)	patient_id, encounter_id	drug_code, description, start_date, stop_date, cost
claims	claim_id (UUID)	patient_id, encounter_id	payer_name, amount_billed, amount_paid, status
vital_signs	vital_id (BIGINT)	patient_id	heart_rate, systolic_bp, diastolic_bp, temp, spo2, timestamp

4.4 Data Cleaning & Preprocessing

- Remove duplicate records across all datasets (Synthea can generate overlapping data)
- Handle nulls: impute missing resolved_date for chronic conditions, fill missing costs with median by encounter type
- Standardize date formats to ISO 8601, normalize drug names using RxNorm mapping
- Validate referential integrity: every encounter references valid patient, every condition references valid encounter
- Type casting: ensure numeric fields (cost, vitals) are proper floats, dates are datetime objects
- Export cleaned datasets as **Parquet files** with appropriate compression (snappy)

4.5 Deliverables

Deliverable	Description	Format
Data Generation Scripts	Synthea config + generate_claims.py + simulate_vitals.py + provider_api.py	Python (.py)
EDA Notebook	15+ visualizations, profiling reports, statistical summaries	Jupyter (.ipynb)
ER Diagram	Normalized schema with all relationships, cardinality, indexes	PNG/PDF (draw.io)
SQL DDL Scripts	CREATE TABLE, indexes, constraints, seed data scripts	SQL (.sql)

Deliverable	Description	Format
Cleaned Parquet Files	All 6 sources cleaned, validated, partitioned	/data/cleaned/*.parquet
Data Quality Report	Issues found, cleaning actions taken, before/after statistics	Markdown (.md)

5. Milestone 2: Data Modeling & Warehouse Design

Modules: Data Modeling + SQL & Database Management + Azure Cloud Services

Duration: Weeks 3-4

5.1 Dimensional Model (Star Schema)

Design a **Star Schema** optimized for healthcare analytical queries:

Fact Tables

Fact Table	Grain	Measures	Foreign Keys
fact_encounters	One row per patient encounter	total_cost, length_of_stay_days, num_conditions, num_medications, readmission_flag	patient_key, provider_key, date_key, payer_key
fact_claims	One row per insurance claim	amount_billed, amount_paid, amount_denied, processing_days	patient_key, encounter_key, payer_key, date_key
fact_vitals	One row per vital signs reading	heart_rate, systolic_bp, diastolic_bp, temperature, spo2, anomaly_flag	patient_key, date_key, time_key

Dimension Tables

Dimension	Type	Key Fields	Notes
dim_patient	SCD Type 2	patient_key, patient_id, name, birth_date, gender, race, city, state, insurance_type, valid_from, valid_to, is_current	Track insurance and address changes
dim_provider	SCD Type 2	provider_key, provider_id, name, specialty, organization, valid_from, valid_to, is_current	Track specialty/org changes
dim_date	Static	date_key, full_date, year, quarter, month, week, day_of_week, is_weekend, is_holiday, fiscal_quarter	Pre-populated calendar
dim_time	Static	time_key, hour, minute, am_pm, shift (morning/afternoon/night)	For vital signs time analysis
dim_diagnosis	Static (Type 1)	diagnosis_key, icd10_code, description, category, body_system	ICD-10 code lookup
dim_payer	Type 1	payer_key, payer_name, payer_type (private/public/self-pay), plan_name	Insurance payer dimension

5.2 Azure Synapse Analytics Setup

- Create Synapse workspace with **dedicated SQL pool** (DW100c for development)
- Implement star schema DDL with distribution strategies:
 - fact_encounters: HASH distributed on patient_key (most common join)
 - fact_claims: HASH distributed on patient_key
 - fact_vitals: HASH distributed on patient_key, partitioned by date
 - All dimensions: REPLICATE distribution (small tables, broadcast to all nodes)
- Configure clustered columnstore indexes on fact tables for analytical query performance
- Set up Azure Data Lake Gen2 as external storage for Synapse Serverless SQL
- Configure Synapse Serverless SQL for ad-hoc querying directly over Bronze/Silver Parquet files

5.3 Analytical SQL Queries (10+)

Write **10+ complex analytical queries** demonstrating advanced SQL skills:

#	Query Name	SQL Techniques Used	Business Question
1	30-Day Readmission Rate	Window functions (LAG, DATEDIFF), CTE, CASE	Which conditions have highest readmission rates?
2	Patient Cost Distribution	NTILE, PERCENTILE_CONT, window aggregation	What is the cost distribution across patient segments?
3	Provider Performance	RANK, DENSE_RANK, AVG OVER partition	Which providers have best patient outcomes?
4	Condition Co-occurrence	Self-join, CROSS APPLY, pivot	Which conditions frequently appear together?
5	Monthly Encounter Trends	DATE_TRUNC, LAG for MoM growth, running total	How are encounter volumes trending?
6	Claim Denial Analysis	GROUP BY GROUPING SETS, HAVING, ratios	What are denial rates by payer and reason?
7	Length of Stay Analysis	DATEDIFF, CASE, MEDIAN, outlier detection	Average LOS by condition and provider?
8	Patient Cohort Retention	Date arithmetic, cohort assignment, pivot	How do patient cohorts retain over time?
9	Top Medications by Cost	SUM, RANK, cumulative percentage (Pareto)	Which medications drive the most cost?
10	Vital Signs Anomaly Summary	WHERE anomaly_flag=1, time patterns, aggregation	When do most vital sign anomalies occur?

5.4 SCD Type 2 Implementation

Implement Slowly Changing Dimensions using **Delta Lake MERGE** operations:

- dim_patient: Track changes in insurance_type, address (city/state), and marital_status
- dim_provider: Track changes in specialty and organization affiliation
- Use surrogate keys (patient_key, provider_key) separate from natural keys (patient_id, provider_id)
- Maintain valid_from, valid_to, and is_current flags for historical tracking

5.5 Deliverables

Deliverable	Description
Star Schema Diagram	Visual diagram with fact/dimension tables, grain statements, relationships, cardinality
Synapse DDL Scripts	CREATE TABLE with distribution, indexing, partitioning strategies documented
SQL Query Notebook	10+ analytical queries with explanations, execution plans, result screenshots
SCD2 Implementation	Delta Lake MERGE scripts for dim_patient and dim_provider
Data Dictionary	All tables, columns, data types, business rules, valid values

6. Milestone 3: Data Pipelines — Batch & Streaming

Modules: Data Pipelines (ETL/ELT) + Big Data (PySpark) + Streaming & Real-Time + Azure Cloud

Duration: Weeks 5-7

6.1 Batch Pipeline — Azure Data Factory + PySpark

6.1.1 Azure Data Factory (ADF) Orchestration

- **Linked Services:** Azure Data Lake Gen2, PostgreSQL, REST API (provider data), Azure Synapse
- **Datasets:** Define source datasets (CSV/JSON on ADLS) and sink datasets (Delta Lake, Synapse tables)
- **Pipelines:**
 - **Pipeline 1 — Full Load:** Initial ingestion of all Synthea CSVs into Bronze layer (ADLS Gen2)
 - **Pipeline 2 — Incremental Load:** Watermark-based incremental ingestion for claims and vitals
 - **Pipeline 3 — API Ingestion:** REST connector to pull provider data from Flask API
- **Triggers:** Scheduled trigger (daily at 2 AM), event-based trigger (new file in landing zone)
- **Error Handling:** Retry policies (3 attempts, 10-minute interval), failure path to dead-letter container, email notifications via Logic Apps

6.1.2 PySpark on Databricks — Bronze to Silver

Cleaning & Validation Layer

- Read raw CSV/JSON from Bronze using `spark.read` with explicit schema enforcement
- **Data Quality Checks:**
 - Null percentage per column (reject if > 30% nulls in critical fields)
 - Schema drift detection (compare incoming schema against expected schema)
 - Duplicate identification using composite keys
 - Range validation: `heart_rate` between 20-250, temperature between 30-45
- **Transformations:**
 - Type casting: dates to `DateType`, costs to `DecimalType(10,2)`, ICD codes to `StringType`
 - Date standardization: all dates to ISO 8601 format
 - String normalization: trim whitespace, standardize gender (M/F/Other), upper-case state codes
 - Deduplication: remove exact duplicates, merge near-duplicates using `patient_id` + `encounter_date`
- **Partitioning:** Partition Silver data by year/month for optimized downstream reads
- **Output:** Write validated data to Silver layer in **Delta Lake format**

6.1.3 PySpark on Databricks — Silver to Gold

Business Logic & Aggregation Layer

- **Surrogate Key Generation:** Generate `patient_key`, `provider_key` using `monotonically_increasing_id()` or SHA-256 hash
- **SCD Type 2 Logic:** Delta Lake MERGE for `dim_patient` and `dim_provider`
 - WHEN MATCHED AND source columns differ THEN expire old row (set `valid_to`, `is_current=false`) AND insert new row
 - WHEN NOT MATCHED THEN insert new dimension record
- **Fact Table Population:**
 - `fact_encounters`: Join encounters + conditions + medications, calculate `length_of_stay`, `readmission_flag` (if same patient within 30 days)
 - `fact_claims`: Join claims with payer data, calculate `amount_denied`, `processing_days`
- **Aggregate Tables:**
 - `monthly_hospital_metrics`: total encounters, avg LOS, readmission rate, total cost
 - `condition_prevalence`: condition counts by demographics (age group, gender, region)
 - `provider_performance`: patient count, avg outcomes, avg cost per provider
- **Write to Synapse:** Push Gold tables via JDBC connector or Synapse Link for Spark

6.2 Streaming Pipeline — Real-Time Vital Signs

6.2.1 Event Hubs Setup

- Create Azure Event Hubs namespace with Standard tier
- Create event hub: `vitals-stream` with 4 partitions, 1-day retention
- Configure Shared Access Policy with Send + Listen permissions

6.2.2 Python Vital Signs Producer

- Script `vitals_producer.py` sends simulated vital signs every 2-5 seconds
- Generates realistic vital sign patterns per patient:
 - Normal readings 85% of the time
 - Mild anomalies 10% (slightly elevated HR, borderline BP)
 - Critical anomalies 5% (tachycardia >150 bpm, hypotension <80/50, fever >39.5C, SpO2 <90%)
- Each message: JSON with `device_id`, `patient_id`, `heart_rate`, `systolic_bp`, `diastolic_bp`, `temperature`, `spo2`, `timestamp`

6.2.3 Spark Structured Streaming Consumer

- Databricks notebook reads from Event Hubs using Spark Structured Streaming
- **Windowed Aggregations:**
 - 5-minute tumbling window: average vitals per patient, min/max ranges
 - 10-minute sliding window: trend detection (is HR increasing over 3 consecutive windows?)

- **Anomaly Detection Logic:**
 - Rule-based alerts: HR > 150 or HR < 40, systolic BP > 180 or < 80, SpO2 < 90%, temp > 39.5C
 - If anomaly detected in any 5-minute window: write alert to alerts Delta table + trigger notification
- **Output:**
 - Append vitals to fact_vitals Delta Lake table (continuous append mode)
 - Write aggregated window results to vitals_summary table
 - Micro-batch write alerts to Synapse for Power BI consumption

6.3 Deliverables

Deliverable	Description
ADF Pipeline Screenshots	All pipeline configs, triggers, linked services, monitoring runs
Databricks Notebooks (Batch)	Bronze-to-Silver and Silver-to-Gold PySpark notebooks with documentation
Streaming Pipeline Code	vitals_producer.py + Spark Structured Streaming consumer notebook
Pipeline Architecture Diagram	End-to-end flow diagram showing all components and data movement
Delta Lake Tables	Bronze/Silver/Gold tables in ADLS Gen2 with proper partitioning
Data Quality Logs	Automated quality check results stored as metadata tables

7. Milestone 4: BI Dashboards & Visualization

Modules: BI & Visualization + Azure Cloud Services

Duration: Week 8

7.1 Power BI Dashboard — Page 1: Executive Summary

- **KPI Cards:** Total Patients, Total Encounters, Average Length of Stay, Overall Readmission Rate, Total Revenue
- **Encounter Volume Trend:** Line chart (daily/weekly/monthly with date slicer)
- **Top 10 Conditions:** Horizontal bar chart by prevalence
- **Cost by Department/Encounter Type:** Stacked bar chart
- **Geographic Map:** Patient distribution by city/state (filled map visual)
- **Slicers:** Date range, encounter type, payer, age group, gender

7.2 Power BI Dashboard — Page 2: Clinical Analytics

- **Readmission Heatmap:** 30-day readmission rates by condition (matrix visual with conditional formatting)
- **Patient Demographics:** Age distribution histogram, gender breakdown, insurance type pie chart
- **Condition Co-occurrence Network:** Treemap or decomposition tree showing related conditions
- **Length of Stay Distribution:** Box plot by encounter type
- **Medication Analysis:** Top prescribed medications, average prescription duration, polypharmacy rates
- **Drill-through:** Click any condition to see detailed patient list, outcomes, and costs

7.3 Power BI Dashboard — Page 3: Operations & Real-Time

- **Bed Occupancy Rate:** Gauge chart (current), trend line (historical)
- **Claim Processing:** Claim status funnel (submitted - approved - denied - appealed - paid)
- **Denial Rate by Payer:** Clustered bar chart with target line
- **Revenue vs Cost:** Waterfall chart showing revenue breakdown
- **Real-Time Vital Signs Panel:** Latest patient vitals from streaming pipeline (DirectQuery to Synapse)
- **Active Alerts:** Card visual showing count of active anomaly alerts in last hour

7.4 DAX Measures

Measure	DAX Logic	Purpose
Readmission Rate	<code>DIVIDE(COUNTROWS(FILTER(fact_encounters, [readmission_flag]=1)), COUNTROWS(fact_encounters))</code>	Track 30-day readmission percentage
Avg Length of Stay	<code>AVERAGE(fact_encounters[length_of_stay_days])</code>	Monitor patient stay duration
YoY Encounter Growth	<code>(([Encounters This Year] - [Encounters Last Year]) / [Encounters Last Year])</code>	Year-over-year volume tracking
Claim Denial Rate	<code>DIVIDE(COUNTROWS(FILTER(fact_claims, [status]="denied")), COUNTROWS(fact_claims))</code>	Insurance denial monitoring
Revenue per Patient	<code>DIVIDE(SUM(fact_claims[amount_paid]), DISTINCTCOUNT(dim_patient[patient_id]))</code>	Patient-level revenue metric
7-Day Moving Avg LOS	<code>AVERAGEX(DATESINPERIOD(dim_date[full_date], LASTDATE(dim_date[full_date]), -7, DAY), [Avg LOS])</code>	Smoothed LOS trend

7.5 Optional: Streamlit Live Dashboard

- Python Streamlit app querying Synapse via pyodbc for real-time vital signs display
- Auto-refreshing every 30 seconds showing: active patient monitors, current vitals, anomaly alerts
- Interactive filters: patient selector, vital sign type, time range
- Alert history timeline with color-coded severity (green/yellow/red)

7.6 Deliverables

Deliverable	Description
Power BI Report (.pbix)	3-page interactive dashboard with slicers, drill-through, tooltips
Dashboard Screenshots	High-resolution exports of each dashboard page
DAX Documentation	All custom measures with business logic explanation
Streamlit App (Optional)	Python app + requirements.txt + deployment instructions

8. Milestone 5: DevOps, Monitoring & Presentation

Modules: DevOps & Monitoring + All Modules (Integration)

Duration: Weeks 9-10

8.1 GitHub Repository Structure

Organize a clean, professional repository:

- `README.md` — Architecture diagram, setup instructions, team contributions, screenshots
- `/src/generators/` — `generate_claims.py`, `simulate_vitals.py`, `provider_api.py`
- `/src/synthea/` — Synthea configuration files and generation scripts
- `/notebooks/` — EDA notebook, Databricks PySpark notebooks (batch + streaming)
- `/sql/` — DDL scripts, analytical queries, stored procedures
- `/pipelines/` — ADF ARM templates, pipeline documentation
- `/dashboards/` — Power BI .pbix file, Streamlit app, screenshots
- `/docs/` — Architecture diagrams, data dictionary, final report
- `/tests/` — pytest unit tests, SQL validation scripts
- `/docker/` — Dockerfile, docker-compose.yml

8.2 CI/CD with GitHub Actions

- **Python Linting:** flake8 on all .py files on every push
- **Unit Tests:** pytest for data generators (validate output schema, data ranges, referential integrity)
- **SQL Validation:** sqlfluff lint on all .sql files
- **Notebook Testing:** papermill for executing Databricks notebooks with test parameters

8.3 Docker Containerization

- **Dockerfile 1:** Data generator service (claims generator + vitals simulator + Flask API)
- **Dockerfile 2:** Streamlit dashboard application
- **docker-compose.yml:** Orchestrate both services + PostgreSQL database for local development
- Include `.env.example` with required environment variables (connection strings, Event Hubs config)

8.4 Monitoring & Alerting

- **Azure Monitor Dashboards:** ADF pipeline success/failure rates, Databricks job health, Synapse query performance
- **Log Analytics:** Centralized logging from all Azure services

- **Alert Rules:**

- Pipeline failure: email notification within 5 minutes
- Data freshness: alert if Gold layer not updated in 2 hours
- Streaming lag: alert if Event Hubs consumer lag > 1000 events
- Vital signs alert rate: notify if anomaly rate > 20% in any 30-minute window

8.5 Data Governance Documentation

- **Role-Based Access Patterns:** Document who should access what (clinical staff vs analysts vs admins)
- **Data Masking Patterns:** Demonstrate column-level masking for patient PII (name, birth_date)
- **Audit Logging:** Track who queries what data and when
- **Data Lineage:** Document data flow from source to dashboard for regulatory compliance

Note: These are documented patterns/designs. Full implementation would require Azure Purview in a production environment.

8.6 Final Presentation

- **Final Report (PDF):** 20-30 pages covering architecture, design decisions, challenges, results, lessons learned
- **Presentation Slides:** 15-20 slides with architecture diagrams, live demo plan, team contributions
- **Live Demo:** Run vitals simulator during presentation, show real-time alerts appearing on dashboard
- **Team Contribution Matrix:** Each member's specific contributions documented

9. Team Role Distribution (6 Members)

Member	Role	Primary Responsibilities	Milestones
#1	Data Engineer Ingestion & Generation	Synthea setup & configuration, generate_claims.py, simulate_vitals.py, Flask provider API, data collection, EDA notebook	M1 (lead) M3 (support)
#2	Data Engineer Modeling & SQL	PostgreSQL schema (3NF), star schema design, ER diagrams, DDL scripts, 10+ analytical SQL queries, SCD2 implementation, data dictionary	M1 (SQL) M2 (lead)
#3	Data Engineer Batch Pipelines	Azure Data Factory setup (linked services, pipelines, triggers), Databricks notebooks for Bronze-to-Silver and Silver-to-Gold batch ETL	M3 batch (lead)
#4	Data Engineer Streaming Pipelines	Event Hubs configuration, Spark Structured Streaming consumer, windowed aggregations, anomaly detection logic, real-time alerts system	M3 stream (lead)
#5	BI Developer & Analyst	Power BI dashboards (3 pages), all DAX measures, data storytelling, optional Streamlit live dashboard, EDA visualization support	M1 (EDA support) M4 (lead)
#6	DevOps & Docs Lead	GitHub repo structure, CI/CD (GitHub Actions), Docker configs, Azure Monitor setup, data governance docs, final report, presentation slides	M5 (lead) All (support)

Collaboration Notes:

- Members #1 and #2 pair closely on M1 (data + schema must align)
- Members #3 and #4 coordinate on M3 (batch Gold output feeds same Synapse tables as streaming output)
- Member #5 starts creating Power BI template during M2 (connect early to Synapse)
- Member #6 supports all milestones with documentation, testing, and repository maintenance
- All 6 members present in the final demo — each demonstrates their specific contribution

10. Timeline (10 Weeks)

Week	Milestone	Key Activities	Checkpoint
1-2	M1: Data Collection & EDA	Generate Synthea data, write custom generators, Flask API, run full EDA, design & implement PostgreSQL schema, clean and export Parquet	EDA notebook review ER diagram approval
3-4	M2: Modeling & Warehouse	Design star schema (3 fact + 6 dim), deploy Synapse, write 10+ analytical queries, implement SCD2, create data dictionary	Schema review Query walkthrough
5-7	M3: Pipelines (Batch + Stream)	Build ADF pipelines (3 pipelines), Databricks ETL (Bronze-Silver-Gold), Event Hubs streaming, anomaly detection, Delta Lake tables	Live pipeline demo Data in Gold layer
8	M4: BI Dashboards	Build Power BI (3 pages), DAX measures, connect to Synapse, optional Streamlit live dashboard	Dashboard walkthrough Team feedback
9-10	M5: DevOps & Presentation	Polish GitHub repo, CI/CD, Docker, Azure Monitor, data governance docs, final report (PDF), presentation slides, rehearsal	Final presentation to instructor

11. Technology Stack

Category	Technologies & Tools
Languages	Python 3.11+, SQL (T-SQL, PostgreSQL), PySpark, DAX
Cloud Platform	Microsoft Azure: Data Lake Gen2, Data Factory, Synapse Analytics, Databricks, Event Hubs, Key Vault, Monitor, Logic Apps
Databases	PostgreSQL (source OLTP), Azure Synapse dedicated SQL pool (OLAP warehouse), SQLite (local dev)
Data Formats	CSV, JSON, Parquet, Delta Lake
Data Generation	Synthea (synthetic patient data), Faker (claims), custom Python simulators (vitals)
Processing	Apache Spark (via Databricks), Spark Structured Streaming, Azure Data Factory
Medical Standards	ICD-10 (diagnosis codes), SNOMED CT (clinical terms), RxNorm (medications)
Visualization	Power BI Desktop + Service, Streamlit (optional), matplotlib, seaborn, plotly
DevOps	Git, GitHub, GitHub Actions (CI/CD), Docker, Docker Compose
Monitoring	Azure Monitor, Log Analytics, Azure Alerts, Application Insights
Python Libraries	pandas, faker, flask, requests, pyodbc, sqlalchemy, pyspark, delta-spark, pytest, flake8, ydata-profiling

12. Final Deliverables Checklist

#	Milestone	Deliverables	Owner(s)
1	M1: Data & EDA	Synthea config + Python generators + Flask API + EDA notebook (15+ charts) + ER diagram + SQL DDL + Cleaned Parquet + Data quality report	#1, #2
2	M2: Modeling	Star schema diagram + Synapse DDL (distribution/indexing) + 10+ SQL queries + SCD2 MERGE scripts + Data dictionary	#2
3	M3: Pipelines	ADF pipelines (3) + screenshots + Databricks notebooks (batch) + Streaming code + Architecture diagram + Delta Lake tables + Quality logs	#3, #4
4	M4: BI	Power BI .pbix (3 pages) + DAX documentation + Dashboard screenshots + Optional Streamlit live dashboard	#5
5	M5: DevOps	GitHub repo (clean structure) + CI/CD + Docker configs + Azure Monitor screenshots + Data governance docs + Final report (PDF) + Slides (15-20)	#6 + all

This proposal provides a complete, portfolio-grade healthcare data engineering project that demonstrates mastery of every concept in the DEPI Microsoft Data Engineer curriculum — from data ingestion with Synthea to real-time vital signs monitoring to clinical BI dashboards.

Build it. Ship it. Demo it.